

Learning Objectives

In Worksheet 4, we constructed the combinational parts of our Mini-ALU (Arithmetical Logical Unit). In the lecture videos, you have since learned more about sequential logic, in particular D latches and D flip-flops. We will now combine these ideas and complete the Mini-ALU.

The primary objective of this worksheet is to understand how combinational and sequential logic work together in a small processor datapath. You will use D flip-flops to store values between operations and combine these storage elements with the circuits developed in Worksheet 4.

By the end of this worksheet, you will have

- built registers for storing 8-bit values and status flags,
- combined four registers into a small register file, with register %0 implemented as a zero register that always contains zero,
- understood that writing to the zero register simply discards the value, while the arithmetic operation itself is still performed normally,
- connected the register file to the adder/subtractor and the status-flag logic developed in Worksheet 4,
- built an instruction decoder that translates an 8-bit machine instruction into the control signals required by the datapath,
- combined the decoder and datapath into the complete Mini-ALU, and
- executed a sequence of machine instructions and observed how registers and status flags change on successive clock edges.

A question to keep in mind: If the result of an arithmetic operation is written to the zero register and therefore discarded, should the status flags still be updated?

Step 1 — Storing the Status Flags

In Worksheet 4, you built the module Status-Flags. Its outputs **ZF**, **CF**, **SF**, and **OF** are determined entirely by its current inputs. The module therefore implements combinational logic.

A processor, however, must remember the status flags produced by an instruction. We will therefore extend the circuit with four D flip-flops. On each *rising edge* of the clock, the currently computed status flags are stored in these flip-flops.

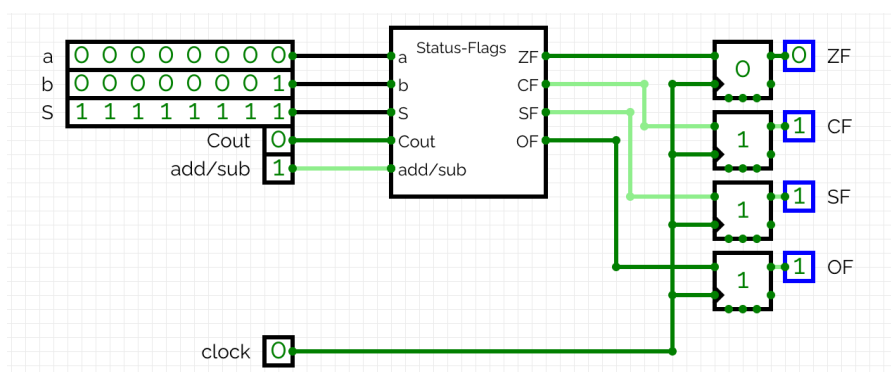
Build a new module Status-Flags-FF with the inputs

a, b, s	8 bits
Cout	1 bit
add/sub	1 bit
clock	1 bit

and the outputs

ZF, CF, SF, OF	each 1 bit
----------------	------------

Use your Status-Flags module from Worksheet 4 to compute the four flags and store each result in a D flip-flop, as illustrated below.



Before continuing: Explain to me or to your neighbor why we use D flip-flops here rather than D latches. In particular, explain the difference between level-triggered and edge-triggered storage. How can the behavior of the two be described mathematically?

SystemVerilog

The primary goal of this course is not to teach you SystemVerilog. Instead, we want to develop a way of thinking about digital hardware that will allow you to work with Hardware Description Languages (HDLs) when you need them. As mathematicians, we are particularly interested in describing precisely what a circuit should do, translating this description into hardware, and understanding the assumptions and limitations of the models we use.

As an example, let us describe the `Status-Flags-FF` module that you have just built in CircuitVerse using SystemVerilog. We will see how the resulting description can first be tested in a simulator and, once it behaves as intended, synthesized and implemented on an FPGA.

```
module status_flags_ff (
    input logic [7:0] a,
    input logic [7:0] b,
    input logic [7:0] s,
    input logic      c_out,
    input logic      add_sub,
    input logic      clock,

    output logic      ZF,
    output logic      CF,
    output logic      SF,
    output logic      OF
);

    always_ff @(posedge clock) begin
        ZF <= ~(s[0] | s[1] | s[2] | s[3] |
               s[4] | s[5] | s[6] | s[7]);

        // ...
    end
endmodule
```

always_ff and Nonblocking Assignments

In Worksheet 4, we used an `always_comb` block to describe combinational logic. The outputs of such a circuit continuously depend on its current inputs. Here we use an `always_ff` block instead. The suffix `ff` stands for *flip-flop*: the block describes sequential logic whose values are stored in flip-flops. The expression

```
always_ff @(posedge clock)
```

specifies when the stored values are updated. `posedge` stands for *positive edge (rising edge)*. Thus, the statements inside the block are evaluated when `clock` changes from 0 to 1.

Also notice the nonblocking assignment operator `<=`:

```
ZF <= ~(s[0] | s[1] | s[2] | s[3] | s[4] | s[5] | s[6] | s[7]);
```

This does not mean that `ZF` continuously follows the expression on the right-hand side. The new value of `ZF` is stored only on a rising edge of clock. Between two rising edges, `ZF` retains its previous value.

Task: Complete the `always_ff` block for `CF`, `SF`, and `OF`. Use the same Boolean expressions that you implemented with logic gates in Worksheet 4.

From mathematical model to physical hardware

In our Boolean model, the expression on the right-hand side appears to be evaluated instantaneously. Real hardware cannot do this. Signals need time to propagate through combinational logic.

If the circuit is built from individual logic gates, this delay is caused by the physical switching behavior of the gates and by the connections between them. On an FPGA, the Boolean expression will typically be mapped to lookup tables (LUTs) and routing resources. These also have nonzero propagation delays. Consequently, the input of a D flip-flop must have reached a stable value before the active clock edge. The clock period therefore cannot be made arbitrarily short.

FPGA synthesis and implementation tools therefore perform timing analysis. They determine whether the combinational paths between flip-flops are fast enough for the requested clock fre-

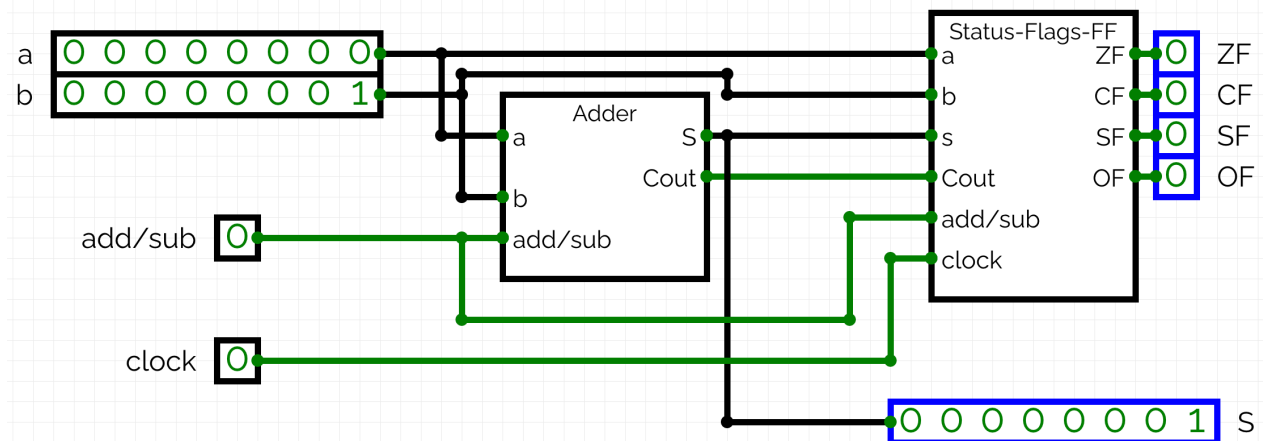
quency. A circuit can be logically correct in a simulator and nevertheless fail to meet the timing requirements of a physical implementation.

For our tiny Mini-ALU this will not be a practical problem. The circuits are small enough that we do not need to worry about timing closure. Nevertheless, it is important to understand why the issue exists.

Step 2 — Combining the Adder and the Status Flags

Before building the register file, let us test our new Status-Flags-FF module together with the Adder from Worksheet 4.

Create a new module called **Adder w/ SF** and connect the two modules as shown below.



The inputs **a**, **b**, and **add/sub** are connected to the **Adder** exactly as in Worksheet 4. The result **s** and carry-out **Cout**, together with the original operands **a** and **b**, are passed through to module **Status-Flags-FF**.

The important difference is that the result **s** is combinational, whereas the status flags are stored.

Experiment

Set **a** = 00000000, **b** = 00000001, and **add/sub** = 1 so that the circuit computes

$$0 - 1 = -1 = s(11111111)_2$$

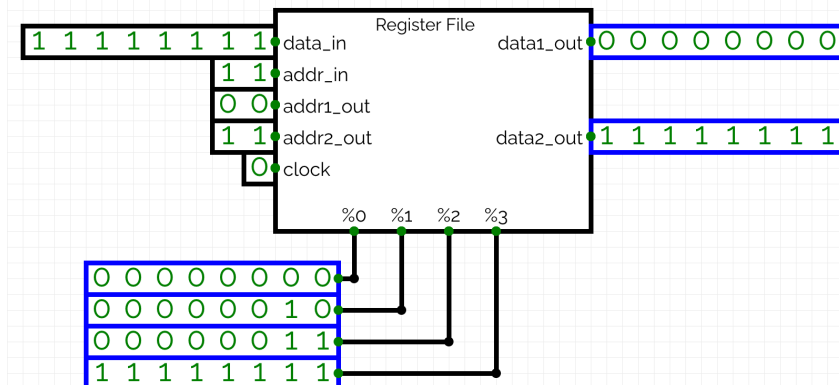
Before generating a rising clock edge, predict the values of **ZF**, **CF**, **SF**, and **OF**.

Task:

- Now generate a rising clock edge and compare your prediction with the stored status flags.
- Then change **a**, **b**, or **add/sub** without generating another rising clock edge. Observe the outputs carefully.
- Explain to me or to your neighbor what you observe and why the circuit behaves this way.

Step 3 — Building the Register File

Our Mini-ALU has four 8-bit registers **%0**, **%1**, **%2**, and **%3**. We will combine them into a module called Register File. The completed module will have the following interface:



The register file provides one input for writing data and two independent outputs for reading data:

- `data_in` is the 8-bit value that can be written to a register.
- `addr_in` selects the destination register. On a rising edge of clock, the value at `data_in` is stored in the selected register.
- `addr1_out` selects a register whose current value is made available at `data1_out`.
- `addr2_out` independently selects a second register whose current value is made available at `data2_out`.

Since the addresses are two bits wide, they select one of the four registers:

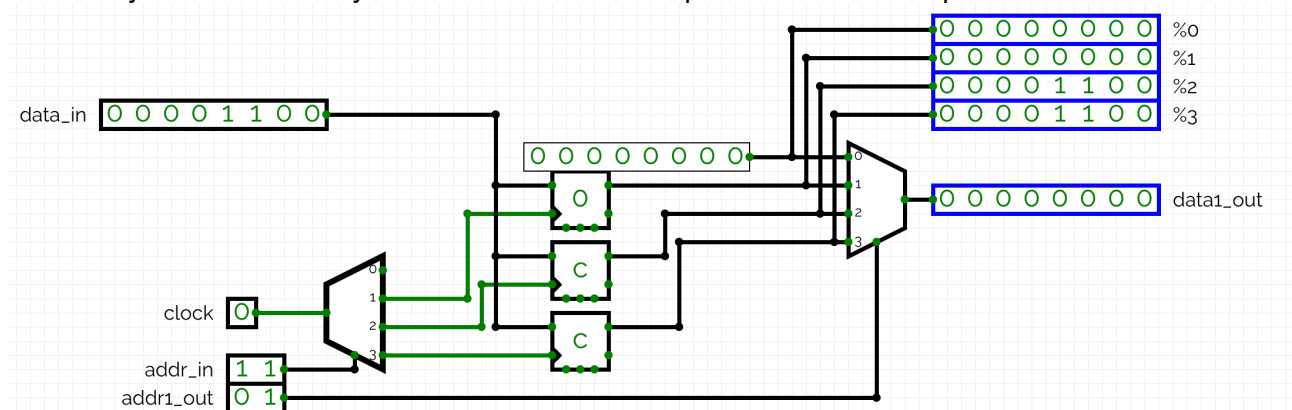
Address	Register
00	%0
01	%1
10	%2
11	%3

Register %0 is special: it is a *zero register*. Values written to %0 are discarded. Reading %0 always produces the 8-bit value 00000000.

The additional outputs %0, %1, %2, and %3 are not needed for computations. They allow us to observe the contents of all registers while debugging the circuit. On the physical Mini-ALU, these outputs are connected to hexadecimal displays, so that the contents of all four registers can be inspected directly.

How does it work?

Most of the register file is already shown in the circuit below. Before completing the circuit, study it carefully and make sure you understand how it implements the description above.



Explain to me or to your neighbor:

- How is the demultiplexer used to select the register that is written on the next rising edge of clock?
- How is the multiplexer used to select the register whose value appears at `data1_out`?
- How is the zero register %0 implemented? What happens if `addr_in = 00` when a rising clock edge occurs?

Task: Complete the register file by adding `addr2_out` and `data2_out`. The second read port should work in exactly the same way as the existing `addr1_out` / `data1_out` read port.

Test your register file: Store different values in %1, %2, and %3. Verify that two registers can be read simultaneously using `addr1_out` and `addr2_out`. Finally, try to write a nonzero value to %0. What happens?

Step 3 – Building the ALU

We now have almost all the major building blocks needed for our Mini-ALU. Let us combine the Register File with the Adder w/SF.

For an addition or subtraction, the second operand always comes from the register selected by `addr2_out`. For the first operand, however, there are two possibilities:

- it can come from the register selected by `addr1_out`, or
- it can be the 8-bit value supplied at `data_in`.

A multiplexer selects between these two possibilities. Its control input is called `use_data1_out`:

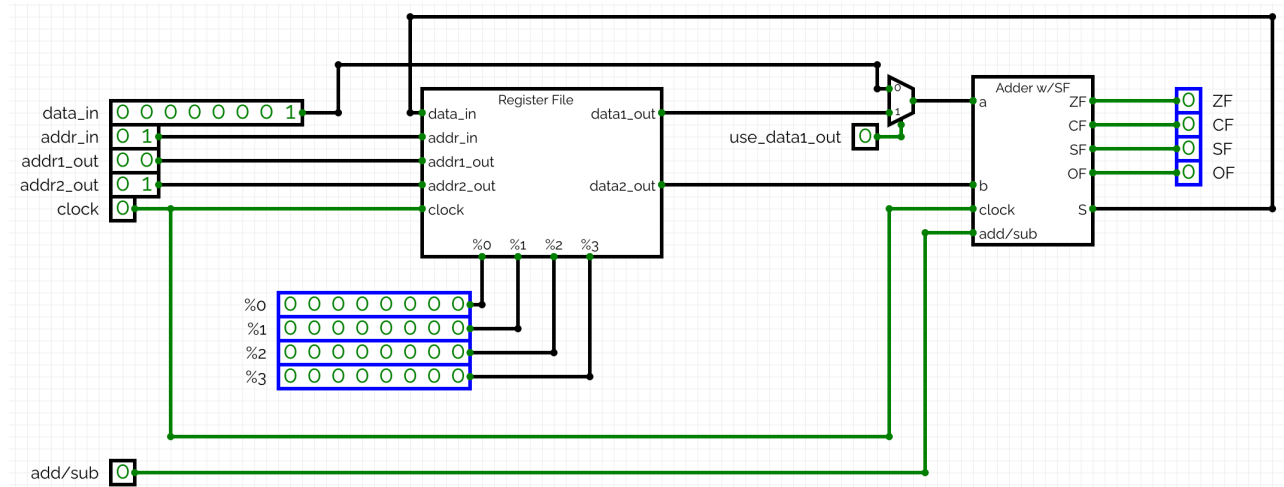
`use_data1_out = 0` -> use `data_in`

`use_data1_out = 1` -> use `data1_out`

When `data_in` is selected, it provides an *immediate value*: a value that comes directly from the instruction rather than from a register.

The output `s` of the adder is connected back to `data_in` of the register file. Therefore, on a rising clock edge, the result of an addition or subtraction is stored in the register selected by `addr_in`.

Build the circuit shown below and turn it into a new module called ALU.



Understanding the data path

Before testing the ALU, trace the paths through the circuit and explain to me or to your neighbor:

- Which path does the first operand take when `use_data1_out = 1`?
- Which path does it take when `use_data1_out = 0`?
- How is the second operand selected?
- How does the result of the adder get back into the register file?
- Which parts of the circuit are combinational and which parts store state?

Task: Controlling the ALU by hand

At the moment, all control signals of the ALU must be set by hand. Let us use this to perform a few operations.

First store 3 in register `%1`. Determine the required values of

<code>data_in</code>	
<code>addr_in</code>	
<code>addr1_out</code>	
<code>addr2_out</code>	
<code>use_data1_out</code>	
<code>add/sub</code>	

and generate a rising clock edge.

Now store 5 in %2

data_in	
addr_in	
addr1_out	
addr2_out	
use_data1_out	
add/sub	

Compute $u(\%3) \leftarrow (u(\%1) + u(\%2)) \bmod 2^8$. Again determine the required control signals before generating the rising clock edge.

data_in	
addr_in	
addr1_out	
addr2_out	
use_data1_out	
add/sub	

Finally compute $u(\%1) \leftarrow (u(\%3) - u(\%2)) \bmod 2^8$.

data_in	
addr_in	
addr1_out	
addr2_out	
use_data1_out	
add/sub	

The zero register

Perform an addition or subtraction, but select %0 as the destination register. Choose the operands such that at least one status flag changes.

Observe %0 and the status flags after the rising clock edge. Explain to me or to your neighbor what you observe and why.

The destination register determines where the result is stored. It does not determine whether the arithmetic operation is performed.

Next step: Decoder

We can now perform all arithmetic operations of our Mini-ALU — but setting six control inputs by hand for every operation is rather inconvenient.

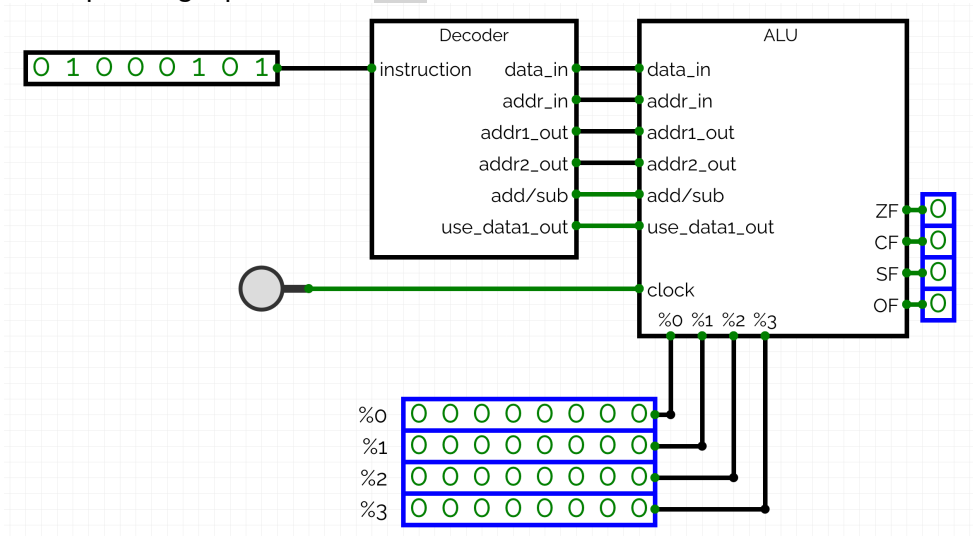
In Worksheet 4, we already defined an 8-bit instruction format containing exactly the information needed for an operation. What we still need is a circuit that translates such an instruction into the control signals of our ALU.

This circuit is called the instruction decoder.

Step 5 – Putting Everything Together

We now have everything we need. The **Decoder** translates an 8-bit instruction into the control signals required by the **ALU**, while the **ALU** performs the operation and stores the result and status flags.

Create a new module called **Mini-ALU** and connect the outputs of the **Decoder** directly to the corresponding inputs of the **ALU**:



The only inputs that remain are the 8-bit instruction and the clock.

That is it. You have built the Mini-ALU.

Running Instructions

From now on, you no longer have to control the individual parts of the ALU manually. Set an 8-bit instruction and generate a rising clock edge. The decoder determines the required control signals, and the ALU executes the instruction.

In Worksheet 4, we considered the following operations only on paper. Execute them now on your Mini-ALU:

Instruction	Operation
00001101	$u(\%1) \leftarrow u(0011) = 3$
01000101	$u(\%1) \leftarrow (u(\%1) + u(0001)) \bmod 2^8$
00001010	$u(\%2) \leftarrow u(0010) = 2$
10011011	$u(\%3) \leftarrow (u(\%1) + u(\%2)) \bmod 2^8$
11011011	$u(\%3) \leftarrow (u(\%1) - u(\%2)) \bmod 2^8$

Finally, execute an arithmetic instruction with **%0** as its destination. Choose the operands such that the operation changes at least one status flag.

Observe the register file and the status flags after the rising clock edge.

Does it matter for the status flags that the result is written to the zero register? Explain your answer from the structure of the Mini-ALU you have built.

Our Mini-ALU can now decode and execute individual machine instructions. What it still cannot do is fetch instructions automatically from memory and execute a sequence of them on its own. We will return to this when we turn our Mini-ALU into a small processor.